

Evaluation Lab, Optional Telemetry, and Memory 2.0

A practical guide to reproducible agent testing, opt-in OTLP traces, workspace search, and user-controlled durable memory.

Area	What it does	Default boundary
Evaluation Lab	Runs reusable Solo or team cases against fixed local fixtures.	Local suites; source workspace is never modified.
Optional Telemetry	Exports completed team-run events as OTLP/HTTP JSON traces.	Off; visible content has a second opt-in.
Workspace Search	Indexes project files for fast lexical or local semantic retrieval.	Local FTS5; optional loopback Ollama embeddings.
Memory 2.0	Stores approved preferences, facts, decisions, procedures, and relationships.	Suggestions wait for approval; approved items are encrypted.

The key mental model

Evaluation measures behavior. Telemetry observes completed runs. Workspace Search retrieves project files. Memory carries approved knowledge forward. They share an interface, but they store different data and have different deletion controls.

Four systems, four jobs

These controls are deliberately separated so testing, observability, project retrieval, and durable memory can be enabled independently.

System	Input	Output	Persists where
Evaluation	Prompt, fixture, assertions, target, budget	Pass/fail, rubric score, latency, tokens, cost, changed paths	Local evaluation and run stores
Telemetry	Completed team-run events	OTLP spans sent to the configured collector	Collector, only when export is enabled
Search index	Eligible project files	Ranked file chunks with paths and line ranges	Per-workspace local SQLite FTS5 index
Durable memory	Reviewed candidate or explicit Remember action	Relevant approved memories plus recall reasons	Encrypted local memory vault

Typical flow



The memory path is separate

Stage	What happens	Who controls it
Suggest	A clear preference, fact, decision, procedure, or relationship enters the Inbox as a candidate.	Agent proposes; it is not yet recallable.
Review	You approve, edit, reject, keep both, or replace an older conflict.	You decide what becomes durable.
Recall	Approved items are ranked against the current request and returned with an explanation.	Agent reads only eligible scopes.
Maintain	Expired facts become stale; conflicts are surfaced for review.	You choose repair or deletion.

No automatic cross-over

Running an evaluation does not create memory. Turning on telemetry does not enable indexing. Indexing a workspace does not approve memories. Each capability has its own switch and data lifecycle.

Local, reproducible agent tests

An evaluation suite is a reusable set of cases that runs a Solo agent or a team against a fixed baseline, then records deterministic and optional subjective results.

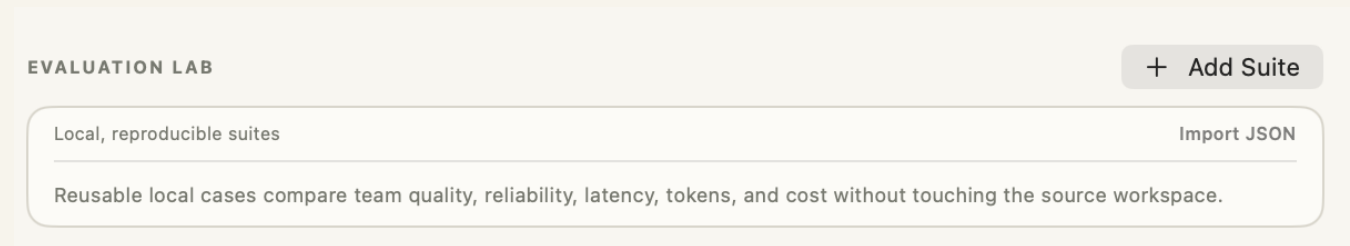


Figure 1. Evaluation Lab before the first suite is created.

- 1

Add Suite

Creates a local suite bound to the current workspace. A suite holds its own name, description, tags, cases, fixture references, and read-only MCP policy.
- 2

Import JSON

Loads a suite definition from a portable JSON file. Validation checks the workspace, case IDs, target and mode values, budgets, and supported assertion types.
- 3

Local, reproducible suites

Cases are saved in the local evaluation store. Git workspaces receive fixed managed fixtures so later runs start from the same baseline rather than today's moving source tree.

Safety boundary

Coding cases run in disposable managed worktrees. Read-only cases can use the source checkout without writing. Computer Control and mutating MCP tools are unavailable; explicitly read-only MCP evidence can be enabled per suite.

What one suite can answer

Question	Evidence
Did the result work?	Required commands, files, paths, JSON checks, output checks, and changed-path rules.
Was the answer good?	Optional blinded reviewer rubric scored from 0 to 100.
Was it reliable?	Pass rate, timeouts, retries, and failure category across repeated cases.
Was it efficient?	Duration, model calls, prompt/completion tokens, estimated cost, and patch size.
Which setup is better?	Run the same fixture against Solo and team targets, then compare like with like.

Designing a strong case

A good case fixes the starting state, states the task precisely, makes success machine-checkable, and caps the resources the run may consume.

Case control	Meaning	Practical use
Mode	Coding or read only	Use coding for file changes; read only for analysis with no workspace write.
Target	Team or Solo	Team uses a selected team manifest; Solo uses one evaluation agent.
Timeout	30 seconds to 120 minutes	Stops a hung case and records timeout as the failure category.
Budget	Jobs, rounds, model calls, concurrent calls, hosted tokens	Prevents an evaluation from expanding beyond its intended cost or complexity.
Rubric	Optional natural-language quality criteria	Evaluated by an eligible reviewer profile with identities hidden.
Passing score	0 to 100	Applied only when a rubric judge produces a subjective score.

Deterministic assertions

Group	Supported checks
Commands	Expected exit code, with a bounded command timeout.
Paths	Path exists or path is absent.
Files	Exact content, substring, or regular-expression match.
Changed paths	Only allowed globs changed, or forbidden globs did not change.
JSON	JSON Pointer equality or minimal JSON Schema validation.
Visible output	Substring or regular-expression match against the final agent response.

How pass/fail is decided

A case passes only if it did not time out, the underlying run completed, every required deterministic assertion passed, and any configured rubric score met the passing threshold. A subjective score cannot override a deterministic failure. Non-required assertions still appear as evidence but do not hard-fail the case.

Running, comparing, and interpreting results

The result record keeps the decision evidence beside the operational metrics, so a faster run is not mistaken for a better run and a high rubric score is not mistaken for a passing test.

Recorded field	How to interpret it
State	Passed or failed after run completion, timeout, deterministic checks, and rubric threshold are combined.
Assertion detail	Per-check pass state, required flag, execution time, and bounded diagnostic output.
Rubric score and reason	Blind reviewer score and explanation; present only when a rubric and reviewer are configured.
Failure category	Timeout, provider/runtime, deterministic assertion, or subjective rubric.
Changed paths and patch bytes	Scope and size of the produced workspace change.
Duration and retries	Latency and recovery cost, useful for reliability comparisons.
Model calls, tokens, estimated cost	Resource usage for the completed case.

A fair comparison checklist

1

Hold the baseline constant

Run both configurations from the same fixture. A current-workspace run is a different experiment.

2

Hold the case and threshold constant

Do not change assertions, rubric, timeout, or budget between compared runs.

3

Separate quality from efficiency

Compare pass rate and rubric first, then latency, retries, tokens, and cost.

4

Repeat noisy cases

One successful sample can hide routing or provider variance. Repeated results reveal reliability.

Results stay isolated

Evaluation output cannot be applied to the source workspace. Successful disposable evaluation checkouts can be cleaned up later; unpinned successful fixtures become eligible for local cleanup after seven days.

Export completed runs with OTLP/HTTP

Telemetry is an observability path for completed team runs. It is disabled by default and sends nothing until export is enabled and an endpoint is configured.

OPTIONAL TELEMETRY

☐ Export completed team runs with OTLP/HTTP

https://collector.example/v1/traces

Authorization header (optional)

☐ Include visible conversation and tool content

Metadata export is off by default. Content has a separate opt-in; credentials remain in Locus's local credential store.

Save Authorization

Figure 2. The telemetry controls are independent of Evaluation Lab.

Control	Behavior
Export completed team runs	After the first durable completion event is processed, Locus posts one OTLP/HTTP JSON trace for that run.
Collector endpoint	Must be an absolute HTTP(S) URL. Remote hosts require HTTPS; loopback hosts may use HTTP.
Authorization header	Optional full header value sent only to this collector. Saved separately in Locus's local credential store.
Include visible content	Adds the sanitized visible event payload to each span. It is a separate, explicit opt-in.

What metadata contains

- Event operation name, agent ID, run ID, sequence, schema version, span timing, and success/error status.
- Provider, model, and job ID when those values are present on the sanitized event.
- Service name and team name as OTLP resource attributes.

Transport and privacy protections

Requests use a 20-second timeout and do not follow redirects. Remote non-HTTPS collectors are rejected. Credentials, secure input, provider signatures, and hidden reasoning are excluded by the run store. Visible conversation and tool content is included only when its separate switch is on, and it remains sanitized before export.

Workspace Search is not durable memory

The advanced panel puts technical controls together, but it manages two stores: a project-file search index and an encrypted memory vault.

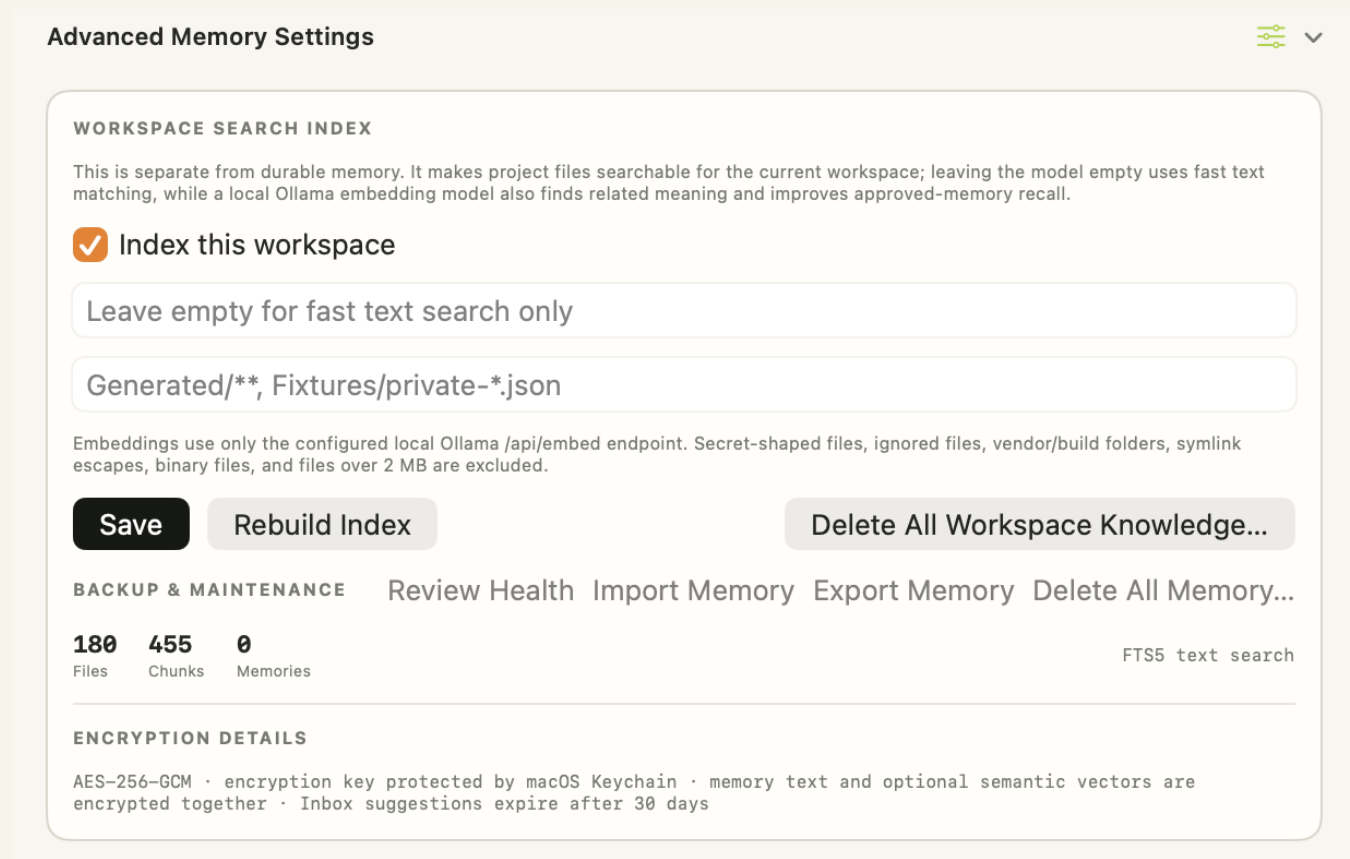


Figure 3. Advanced Memory Settings with FTS5 text search active and no approved memories.

On-screen metric	What it counts
Files	Eligible project files currently represented in the workspace search index.
Chunks	Searchable text segments made from those files.
Memories	Approved durable memory visible for the current workspace and selected memory owner.
FTS5 text search	The embedding model is empty, so retrieval is lexical only.

Reading the screenshot

180 files and 455 chunks means project search is populated. 0 memories means no approved durable memories are currently visible. It does not mean the index is empty, and it does not reveal whether candidates are waiting in the Inbox.

Fast local retrieval with optional semantics

The index makes eligible project text discoverable for the current canonical workspace. It is designed to fail safely back to text search when local embeddings are unavailable.

Control	Detailed behavior
Index this workspace	Enables indexing for this workspace. Disabling it leaves durable memory as a separate system.
Embedding model	Leave empty for SQLite FTS5 lexical search. Enter a local Ollama model to add vector similarity and improve approved-memory semantic recall.
Additional exclusions	Comma-separated glob patterns such as Generated/** or Fictures/private-*.json.
Save	Stores enabled state, model, local Ollama host, and exclusion patterns.
Rebuild Index	Reprocesses eligible files and refreshes the stored chunks; useful after model or exclusion changes.
Delete All Workspace Knowledge	After confirmation, deletes the workspace search index and workspace-scoped durable memory. Project files and chat transcripts remain.

What never enters the index

- Files matching Git ignore rules or your additional exclusion globs.
- Secret-shaped filenames, common key/certificate/environment-secret names, and hidden/vendor/build paths.
- Binary files and files larger than 2 MB.
- Symlinks and paths that could escape the canonical workspace.

Local embedding boundary

Semantic indexing uses only the configured loopback Ollama /api/embed endpoint. If embedding fails, lexical search stays available. Search results are evidence with relative paths and line ranges; they do not become instructions and they are not automatically saved as memory.

Retrieval path

Step	Action
1. Scan	Resolve the workspace, honor ignore/exclusion/safety rules, hash eligible content.
2. Chunk	Store bounded text segments with source path and line metadata.
3. Index	Always add FTS5 terms; optionally add local Ollama vectors.
4. Retrieve	Return the most relevant source chunks and preserve their provenance.

Approved, scoped, explainable, and encrypted

Durable memory is for stable knowledge the user wants carried forward. Suggestions are candidates, not facts, until they are approved.

Dimension	Options and behavior
Kinds	Preference, fact, decision, procedure, relationship.
Scopes	Personal is shared across workspaces; workspace is bound to the canonical project; agent follows the selected member.
Status	Candidate waits in Inbox; approved is eligible for recall. Candidates expire after 30 days.
Quality metadata	Confidence, tags, provenance, valid-from/valid-until, pin, stale state, last-used time, use count, and revision.
Feedback	Helpful, ignored, or incorrect; incorrect marks the item stale.
Conflicts	Same-topic alternatives are surfaced. Keep Both preserves both; Replace marks older conflicts stale and links supersession.

How recall is ranked

- 1

Eligibility

Only approved items in the active personal, workspace, and selected-agent scopes are considered. Future-valid items are skipped.
- 2

Relevance

Exact phrase and term matches are combined with optional local semantic similarity. Very weak matches are filtered out.
- 3

Trust and freshness

Pinned items, confidence, and recency add weight. Expired and stale items are strongly down-ranked.
- 4

Explanation

Each recalled item reports why it matched, such as exact phrase, matching terms, semantic similarity, pinned, and confidence.

Encryption boundary

Memory payloads and optional semantic vectors are encrypted together with AES-256-GCM. The 256-bit master key lives in the macOS login Keychain and is handed to the local backend only through its startup pipe; it is not stored in app preferences, process arguments, or environment variables.

Portable exports are deliberately readable

The local database is encrypted, but Export Memory creates a human-readable JSON file so it can be inspected or moved. Protect that exported file like any other sensitive document.

Choose the right action

The maintenance controls are explicit because the index, Inbox, approved memory, and external telemetry collector have different lifecycles.

Action	Removes or changes	Leaves intact
Review Health	Marks expired approved memories stale and reports conflicts for human review.	Index, files, chats, and non-expired memory.
Import Memory	Adds validated memories from a readable Locus Memory JSON export.	Existing index and source project.
Export Memory	Writes a readable JSON copy selected through the save panel.	Encrypted vault remains unchanged.
Delete Workspace Knowledge	Workspace index plus workspace-scoped candidate and approved memory.	Project files, chats, personal memory, agent memory.
Delete All Memory	Visible personal, workspace, and selected-agent candidate/approved memory.	Project index, project files, chats.
Disable telemetry	Stops future automatic OTLP export.	Anything already retained by the external collector.

Recommended starting configurations

Situation	Recommended setup
Personal local development	Index on, embedding model empty, telemetry off, approve only durable project conventions.
Large codebase with local Ollama	Index on, a small local embedding model, explicit generated/vendor exclusions, periodic rebuild after model changes.
Team comparison	One fixed suite, identical budgets and thresholds, repeated Solo and team runs, telemetry optional.
Sensitive workspace	Text-only index, aggressive exclusions, telemetry off, export memory only to an encrypted destination.
Internal observability	Metadata telemetry first; enable visible content only after collector retention and access controls are reviewed.

A good default

Start with FTS5 text search, telemetry off, and an empty Memory Inbox. Add local embeddings when meaning-based retrieval is worth the local compute. Approve memories sparingly: durable decisions and conventions are more useful than transient task details.